

Olist Brazilian E-Commerce — End-to-End Analytics Portfolio Project

A Recruiter-Ready Guide for Summer 2026 Data Analyst Internship Applications

Your background: Python, pandas, NumPy, scikit-learn, statistics, messy real-world data

What you're building toward: SQL fluency, ETL pipelines, business analytics, cohort retention, RFM segmentation, KPI dashboards

Table of Contents

- 1. [Phase 1 — Project Planning & Architecture](#)
- 2. [Phase 2 — Environment Setup](#)
- 3. [Phase 3 — Dataset Understanding](#)
- 4. [Phase 4 — ETL / Data Cleaning Pipeline](#)
- 5. [Phase 5 — Exploratory Data Analysis \(EDA\)](#)
- 6. [Phase 6 — SQL Analytics \(15-20 Queries\)](#)
- 7. [Phase 7 — Cohort Retention Analysis](#)
- 8. [Phase 8 — RFM Customer Segmentation](#)
- 9. [Phase 9 — KPI Design](#)
- 10. [Phase 10 — Dashboard Planning](#)
- 11. [Phase 11 — Power BI / Tableau Dashboard Build](#)

[](#)

Phase 1 — Project Planning & Architecture

The Big Picture: What You're Actually Building

This is not a "data science project." It's a **business analytics project** — you're simulating the work of an analyst embedded in a real e-commerce company. The deliverable is actionable intelligence that helps business teams make decisions.

Your pipeline looks like this:



Each arrow represents a job skill that recruiters look for:

Step	Skill Signal
Raw CSVs → ETL	Data wrangling, cleaning, pipeline thinking
ETL → PostgreSQL	Database design, SQL, production workflows
PostgreSQL → SQL Analysis	Analytical SQL, business thinking
SQL → Python Analysis	Pandas, cohort analysis, segmentation
Python → Dashboard	Visualization, stakeholder communication

Why Each Step Matters in Real Analyst Jobs

ETL Pipeline: Every company ingests data from multiple sources (CRMs, transactional databases, marketing platforms). Knowing how to clean and load data is day-one work at most analytics teams. Showing you can do this signals you won't need hand-holding.

SQL in a Database (not just pandas): Pandas can do most of what SQL does — but in industry, data lives in databases (Redshift, BigQuery, Snowflake, PostgreSQL). Analysts write SQL daily. Your ML background is strong; SQL is your gap, and this project closes it.

Cohort Retention Analysis: This is a standard interview question and a staple deliverable at every SaaS, e-commerce, or subscription business. "How many customers who bought in January came back in February?" — this is cohort analysis. Being able to build this from scratch sets you apart.

RFM Segmentation: Marketing and CRM teams run campaigns based on customer segments. Recency, Frequency, Monetary scoring is a classic, well-understood framework. Interviewers know it; you should too.

Dashboard: The dashboard is what stakeholders actually see. Knowing how to tell a story with data — not just produce charts — is a soft skill that differentiates senior analysts.

What Recruiters Expect from This Kind of Project

- Clean, version-controlled code on GitHub (not a messy notebook)
- Evidence of **business thinking** (not just data manipulation)
- SQL queries that answer real business questions
- Visualizations that tell a story, not just describe data
- A README that explains what you found, not just what you did

Tool Recommendations for Your Situation

PostgreSQL vs DuckDB:

Use **PostgreSQL** for this project. Here's why: PostgreSQL is what you'll encounter in internships. It's the industry standard for transactional and analytical workloads at mid-size companies. DuckDB is excellent for local analytics and is growing in popularity, but if an interviewer asks "what database did you use?" and you say PostgreSQL, they immediately understand your skill level. DuckDB is also harder to demonstrate to a non-technical recruiter.

That said, DuckDB is worth knowing — it's faster for local analytical queries and requires no server setup. After this project, consider re-running your SQL queries in DuckDB to learn it as a secondary skill.

Tableau vs Power BI:

Use **Power BI** if you're targeting Canada/US corporate internships (banks, consulting, retail, tech). Use **Tableau** if you're targeting data/tech-forward companies. Power BI has a free Desktop version; Tableau Public is free but publishes your work online. Both are fine — pick one and go deep. This guide uses Power BI but annotates Tableau equivalents.

Jupyter vs VSCode:

Use **VSCode with Jupyter integration**. Here's the workflow: write your ETL scripts as `.py` files (looks professional, version-controls cleanly), write your EDA and analysis in `.ipynb` notebooks inside VSCode. This gives you the best of both worlds. Avoid pure Jupyter Lab for production code — it encourages messy, non-modular notebooks.

Phase 2 — Environment Setup

PostgreSQL Installation

macOS:

```
bash

brew install postgresql@15
brew services start postgresql@15
echo 'export PATH="/opt/homebrew/opt/postgresql@15/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Ubuntu/Debian (including WSL2):

```
bash
```

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

Windows: Download the installer from <https://www.postgresql.org/download/windows/> — use the graphical installer and install pgAdmin at the same time.

Create Your Project Database

```
bash

# Connect as postgres superuser
psql -U postgres

# Inside psql:
CREATE DATABASE olist_analytics;
CREATE USER olist_user WITH PASSWORD 'your_secure_password';
GRANT ALL PRIVILEGES ON DATABASE olist_analytics TO olist_user;
\q
```

pgAdmin Setup

Download pgAdmin 4 from <https://www.pgadmin.org/download/>. After installation:

1. Right-click Servers → Register → Server
2. Name:
3. Connection tab: Host , Port , Database , Username

pgAdmin gives you a GUI to inspect tables, run queries, and verify your ETL loaded data correctly.

Python Environment

```
bash

# Create a virtual environment in your project root
```

```
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate

# Install all dependencies
pip install -r requirements.txt
```

requirements.txt

```
pandas==2.2.1
numpy==1.26.4
sqlalchemy==2.0.28
psycopg2-binary==2.9.9
matplotlib==3.8.3
seaborn==0.13.2
plotly==5.19.0
jupyter==1.0.0
python-dotenv==1.0.1
```

Recommended Project Folder Structure

```
olist-analytics/
├── README.md
├── requirements.txt
├── .env                    # DB credentials (never commit this)
├── .gitignore
├── data/
│   ├── raw/               # Original CSVs – never touch these
│   │   ├── olist_customers_dataset.csv
│   │   ├── olist_orders_dataset.csv
│   │   └── ... (all 9 CSVs)
│   └── processed/         # Cleaned CSVs (optional intermediate step)
├── etl/
│   ├── 01_load_raw.py     # Load raw CSVs
│   ├── 02_clean_customers.py
│   ├── 02_clean_orders.py
│   ├── 02_clean_order_items.py
│   ├── 02_clean_reviews.py
│   ├── 03_create_schema.sql # SQL DDL – table definitions
│   └── 04_load_to_postgres.py # Final load into PostgreSQL
├── analysis/
│   └── 01_analysis.py
```

```
|   |   | 01_eda.ipynb           # All SQL queries, organized and commented
|   |   | 02_sql_analysis.sql      # All SQL queries, organized and commented
|   |   | 03_cohort_retention.ipynb
|   |   | 04_rfm_segmentation.ipynb
|
|   | dashboard/
|   |   | olist_dashboard.pbix      # Power BI file
|   |   | dashboard_notes.md       # Notes on design decisions
|   |
|   | outputs/
|   |   | figures/                  # Saved charts
|   |   | exports/                  # CSVs exported for dashboard
```

Why this structure matters: In a real analytics team, your ETL scripts run on a schedule (Airflow, cron, etc.), your analysis notebooks are shared with teammates, and your SQL queries are reviewed in PRs. A clean structure signals you understand production workflows, not just notebooks.

.env file (never commit to git)

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=olist_analytics
DB_USER=olist_user
DB_PASSWORD=your_secure_password
```

.gitignore

```
venv/
.env
*.pyc
__pycache__/
data/raw/
outputs/
.DS_Store
```

Phase 3 — Dataset Understanding

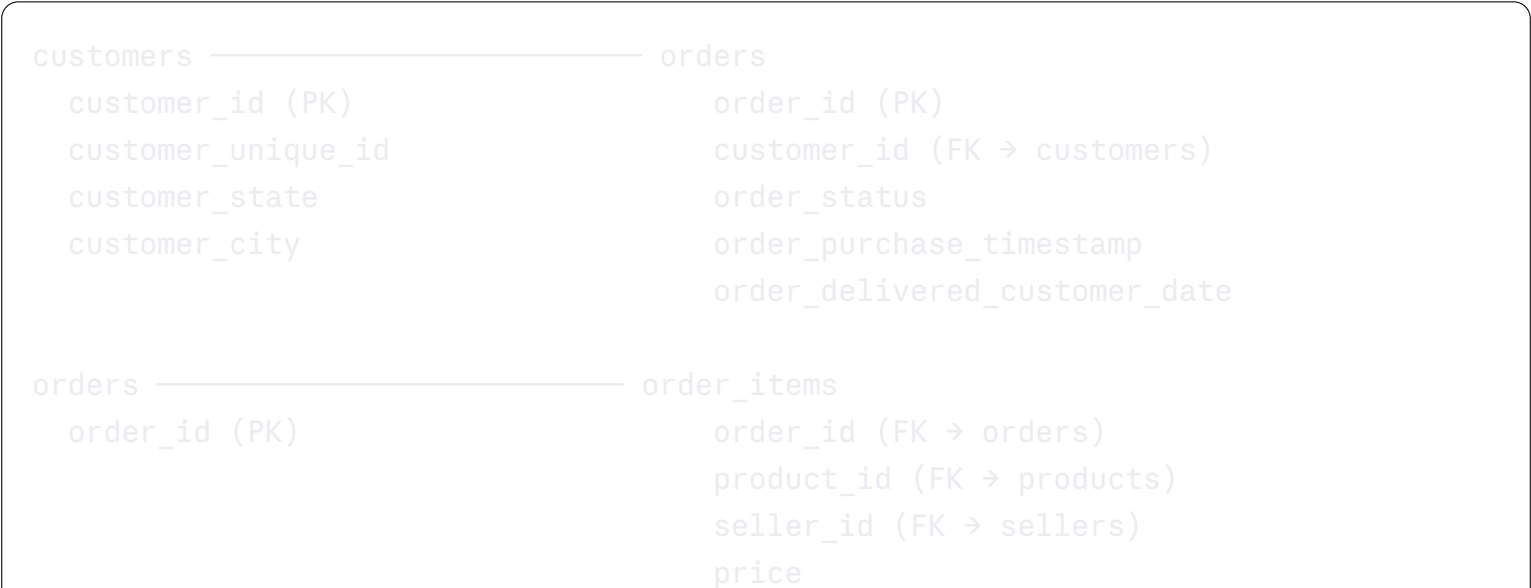
The Full Olist Dataset

You uploaded `olist_customers_dataset.csv`. The complete Olist dataset has **9 tables** — download the full dataset from Kaggle: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

The 9 tables and their business meaning:

Table	Rows (approx)	Business Purpose
<code>olist_customers_dataset.csv</code>	99,441	Customer master — unique IDs, location
<code>olist_orders_dataset.csv</code>	99,441	Order header — status, timestamps
<code>olist_order_items_dataset.csv</code>	112,650	Line items — products, sellers, prices
<code>olist_order_payments_dataset.csv</code>	103,886	Payment method and installment info
<code>olist_order_reviews_dataset.csv</code>	99,224	Customer review scores and text
<code>olist_products_dataset.csv</code>	32,951	Product catalog with categories
<code>olist_sellers_dataset.csv</code>	3,095	Seller master — IDs and location
<code>olist_geolocation_dataset.csv</code>	1,000,163	ZIP → lat/lng mapping
<code>product_category_name_translation.csv</code>	71	Portuguese → English category names

Entity-Relationship Overview



freight_value



Key Design Decision: customer_id vs customer_unique_id

This is a critical business insight hiding in the schema. The dataset has **two customer ID columns**:

- customer_id**: Unique per *order*. The same physical customer can have multiple **customer_id** values if they ordered more than once.
- customer_unique_id**: Truly unique per *person*. This is the correct field for customer-level analysis (retention, RFM, LTV).

This is a real-world data modeling pattern. Companies often generate a new session ID per purchase event — the "unique" identifier has to be derived or stored separately. If you use **customer_id** for retention analysis, you'll get misleadingly low retention rates. Always use **customer_unique_id** for customer-level metrics.

Analytical Dimensions and Measures

Dimensions (what you slice by):

- Time: purchase month/year, delivery month
- Geography: customer_state, seller_state
- Product: category (English), product_id
- Customer segment: RFM tier, state
- Order status: delivered, canceled, shipped

Measures (what you aggregate):

- Revenue: **price + freight_value** (total order value)

- Volume: count of orders, count of items
 - Delivery performance: $\frac{\text{order_delivered_customer_date} - \text{order_estimated_delivery_date}}{\text{order_estimated_delivery_date}}$
 - Customer satisfaction: average review score
 - Retention: repeat purchase rate (requires $\text{customer_unique_id}$)
-

Phase 4 — ETL / Data Cleaning Pipeline

Why ETL Matters in Real Analytics Teams

In industry, raw data is never clean. An analyst who can build a reliable ETL pipeline — one that catches errors, handles edge cases, and loads cleanly into a database — is far more valuable than one who only does analysis. This section teaches you to think like a data engineer as well as an analyst.

etl/03_create_schema.sql — Database Schema (DDL)

Run this first in pgAdmin or psql to create your tables before loading data.

```
sql

-- =====
-- OLIST ANALYTICS – TABLE DEFINITIONS
-- Run this script once before loading data
-- =====

CREATE TABLE IF NOT EXISTS customers (
    customer_id          VARCHAR(40) PRIMARY KEY,
    customer_unique_id   VARCHAR(40) NOT NULL,
    customer_zip_code_prefix INTEGER,
    customer_city         VARCHAR(100),
    customer_state        CHAR(2)
);

CREATE TABLE IF NOT EXISTS orders (
    order_id              VARCHAR(40) PRIMARY KEY,
    customer_id            VARCHAR(40) REFERENCES customers(customer_id),
    order_status           VARCHAR(20),
    order_purchase_timestamp TIMESTAMP,
    order_approved_at      TIMESTAMP,
    order_delivered_carrier_date TIMESTAMP,
```

```
order_delivered_customer_date    TIMESTAMP,  
order_estimated_delivery_date    TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS order_items (  
  
    order_id            VARCHAR(40) REFERENCES orders(order_id),  
    order_item_id       INTEGER,  
    product_id          VARCHAR(40) REFERENCES products(product_id),  
    seller_id           VARCHAR(40) REFERENCES sellers(seller_id),  
    shipping_limit_date  TIMESTAMP,  
    price               NUMERIC(10,2),  
    freight_value        NUMERIC(10,2),  
    PRIMARY KEY (order_id, order_item_id)  
);
```

```
CREATE TABLE IF NOT EXISTS order_payments (  
  
    order_id            VARCHAR(40) REFERENCES orders(order_id),  
    payment_sequential  INTEGER,  
    payment_type        VARCHAR(30),  
    payment_installments INTEGER,  
    payment_value        NUMERIC(10,2),  
    PRIMARY KEY (order_id, payment_sequential)  
);
```

```
CREATE TABLE IF NOT EXISTS order_reviews (  
  
    review_id           VARCHAR(40) PRIMARY KEY,  
    order_id            VARCHAR(40) REFERENCES orders(order_id),  
    review_score         SMALLINT CHECK (review_score BETWEEN 1 AND 5),  
    review_comment_title TEXT,  
    review_comment_message TEXT,  
    review_creation_date  TIMESTAMP,  
    review_answer_timestamp TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS products (  
  
    product_id          VARCHAR(40) PRIMARY KEY,  
    product_category_name VARCHAR(100),  
    product_name_length  INTEGER,  
    product_description_length INTEGER,  
    product_photos_qty   INTEGER,  
    product_weight_g      NUMERIC(8,2),  
    product_length_cm     NUMERIC(6,2),  
    product_height_cm     NUMERIC(6,2),  
    product_width_cm      NUMERIC(6,2)  
);
```

```

CREATE TABLE IF NOT EXISTS sellers (
    seller_id          VARCHAR(40)  PRIMARY KEY,
    seller_zip_code_prefix  INTEGER,
    seller_city         VARCHAR(100),
    seller_state        CHAR(2)
);

CREATE TABLE IF NOT EXISTS geolocation (
    geolocation_zip_code_prefix  INTEGER,
    geolocation_lat              NUMERIC(10,6),
    geolocation_lng              NUMERIC(10,6),
    geolocation_city             VARCHAR(100),
    geolocation_state            CHAR(2)
);

CREATE TABLE IF NOT EXISTS product_category_translation (
    product_category_name        VARCHAR(100)  PRIMARY KEY,
    product_category_name_english  VARCHAR(100)
);

```

etl/04_load_to_postgres.py — ETL Pipeline

```
python
```

```

"""
etl/04_load_to_postgres.py

Full ETL pipeline: load CSVs, clean, validate, and write to PostgreSQL.

```

Design principles:

- Each table has its own cleaning function for modularity
- Validation checks run before loading (fail-fast pattern)
- All operations are idempotent — safe to re-run
- Logging to stdout so you can see what's happening

```

"""

import os
import logging
import pandas as pd
import numpy as np
from sqlalchemy import create_engine, text
from dotenv import load_dotenv

```

```
# — Configuration —
```

```
# Configuration
```

```
load_dotenv()
```

```
logging.basicConfig(
```

```
    level=logging.INFO,
```

```
    format="%(asctime)s [%(levelname)s] %(message)s",
```

```
    datefmt="%H:%M:%S"
```

```
)
```

```
log = logging.getLogger(__name__)
```

```
RAW_DIR = "data/raw"
```

```
DB_URL = (
```

```
    f"postgresql+psycopg2://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}@"
```

```
    f"@{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB_NAME')}"
```

```
)
```

```
# — Helper Functions
```

```
def load_csv(filename: str, **kwargs) -> pd.DataFrame:
```

```
    """Load a raw CSV and log its shape."""
```

```
    path = os.path.join(RAW_DIR, filename)
```

```
    df = pd.read_csv(path, **kwargs)
```

```
    log.info(f"Loaded {filename}: {df.shape[0]:,} rows × {df.shape[1]} cols")
```

```
    return df
```

```
def validate_no_null_pk(df: pd.DataFrame, pk_col: str, table_name: str) -> None:
```

```
    """Fail loudly if primary key column contains nulls."""
```

```
    nulls = df[pk_col].isnull().sum()
```

```
    if nulls > 0:
```

```
        raise ValueError(f"[{table_name}] Primary key '{pk_col}' has {nulls} NULL values")
```

```
    log.info(f"[{table_name}] PK validation passed: no NULLs in '{pk_col}'")
```

```
def validate_no_duplicate_pk(df: pd.DataFrame, pk_cols: list, table_name: str) -> None:
```

```
    """Fail loudly if primary key columns contain duplicates."""
```

```
    dupes = df.duplicated(subset=pk_cols).sum()
```

```
    if dupes > 0:
```

```
        raise ValueError(f"[{table_name}] {dupes} duplicate rows on PK {pk_cols}")
```

```
    log.info(f"[{table_name}] Duplicate check passed on {pk_cols}")
```

```
def write_to_postgres(df: pd.DataFrame, table_name: str, engine) -> None:
```

```
"""Write a DataFrame to PostgreSQL, replacing existing data."""
df.to_sql(table_name, engine, if_exists="replace", index=False, method="multi",
log.info(f"Written '{table_name}' to PostgreSQL: {len(df):,} rows")
```

— Cleaning Functions – one per table

```
def clean_customers(df: pd.DataFrame) -> pd.DataFrame:
```

```
    """
```

```
    Customers table cleaning.
```

```
    Key issue: customer_city names have inconsistent casing and
    trailing whitespace from the source system. We normalize them
    because city names are used as join keys with geolocation.
```

```
    """
```

```
    df = df.copy()
```

```
    # Normalize text columns
```

```
    df["customer_city"] = df["customer_city"].str.strip().str.lower()
```

```
    df["customer_state"] = df["customer_state"].str.strip().str.upper()
```

```
    # Drop exact duplicates (same customer_id appearing twice)
```

```
    before = len(df)
```

```
    df = df.drop_duplicates(subset=["customer_id"])
```

```
    if len(df) < before:
```

```
        log.warning(f"Dropped {before - len(df)} duplicate customer_id rows")
```

```
    validate_no_null_pk(df, "customer_id", "customers")
```

```
    validate_no_duplicate_pk(df, ["customer_id"], "customers")
```

```
    return df
```

```
def clean_orders(df: pd.DataFrame) -> pd.DataFrame:
```

```
    """
```

```
    Orders table cleaning.
```

```
    Key issues:
```

1. Timestamp columns arrive as strings – must parse to datetime
2. Some orders have NULL delivery dates (undelivered orders) – this is expected and valid, not a data quality problem. We keep them.
3. A small number of orders have delivered_date < purchase_date – these are data entry errors; we nullify the delivery date.

```
    """
```

```
    df = df.copy()
```

```

timestamp_cols = [
    "order_purchase_timestamp",
    "order_approved_at",
    "order_delivered_carrier_date",
    "order_delivered_customer_date",
    "order_estimated_delivery_date",
]

for col in timestamp_cols:
    df[col] = pd.to_datetime(df[col], errors="coerce")
    nulls = df[col].isnull().sum()
    if nulls > 0:
        log.info(f"  orders.{col}: {nulls:,} NULLs (expected for undelivered orders)")

# Flag and nullify impossible delivery dates
bad_delivery = (
    df["order_delivered_customer_date"].notna()
    & (df["order_delivered_customer_date"] < df["order_purchase_timestamp"])
)
if bad_delivery.sum() > 0:
    log.warning(f"Nullifying {bad_delivery.sum()} impossible delivery dates")
    df.loc[bad_delivery, "order_delivered_customer_date"] = pd.NaT

# Normalize status strings
df["order_status"] = df["order_status"].str.strip().str.lower()

validate_no_null_pk(df, "order_id", "orders")
validate_no_duplicate_pk(df, ["order_id"], "orders")

# Log order status distribution (useful sanity check)
status_counts = df["order_status"].value_counts()
log.info(f"Order status distribution:\n{status_counts.to_string()}")

return df

def clean_order_items(df: pd.DataFrame) -> pd.DataFrame:
    """
    Order items table cleaning.

    Key issues:
    1. price and freight_value should always be positive
    2. Composite PK: (order_id, order_item_id) must be unique
    """

```

```

df = df.copy()

df["shipping_limit_date"] = pd.to_datetime(df["shipping_limit_date"], errors="co

# Flag negative prices – should not exist
neg_price = (df["price"] < 0).sum()
neg_freight = (df["freight_value"] < 0).sum()
if neg_price > 0:
    log.warning(f"Found {neg_price} rows with negative price – setting to NaN")
    df.loc[df["price"] < 0, "price"] = np.nan
if neg_freight > 0:
    log.warning(f"Found {neg_freight} rows with negative freight_value – setting
    df.loc[df["freight_value"] < 0, "freight_value"] = np.nan

# Derived column: total item value (useful for downstream analysis)
# We add this here so every downstream query can use it
df["total_value"] = df["price"].fillna(0) + df["freight_value"].fillna(0)

validate_no_duplicate_pk(df, ["order_id", "order_item_id"], "order_items")

return df

def clean_order_payments(df: pd.DataFrame) -> pd.DataFrame:
    """Order payments: validate payment values are positive."""
    df = df.copy()

    neg_payment = (df["payment_value"] < 0).sum()
    if neg_payment > 0:
        log.warning(f"Found {neg_payment} negative payment values – reviewing")

    validate_no_duplicate_pk(df, ["order_id", "payment_sequential"], "order_payments")

    return df

def clean_order_reviews(df: pd.DataFrame) -> pd.DataFrame:
    """
    Reviews: review_score must be 1-5.
    Timestamps: parse and validate order.
    """
    df = df.copy()

    df["review_creation_date"] = pd.to_datetime(df["review_creation_date"], errors="
    df["review_answer_timestamp"] = pd.to_datetime(df["review_answer_timestamp"], e

```

```

# Validate score range
invalid_scores = ~df["review_score"].between(1, 5)
if invalid_scores.sum() > 0:
    log.warning(f"Found {invalid_scores.sum()} review scores outside [1,5] - set to nan")
    df.loc[invalid_scores, "review_score"] = np.nan

df["review_score"] = df["review_score"].astype("Int64") # nullable integer

# Drop duplicate review_ids (keep first)
df = df.drop_duplicates(subset=["review_id"])

return df


def clean_products(df: pd.DataFrame) -> pd.DataFrame:
    """Products: standardize category names."""
    df = df.copy()
    df["product_category_name"] = df["product_category_name"].str.strip().str.lower()
    validate_no_null_pk(df, "product_id", "products")
    validate_no_duplicate_pk(df, ["product_id"], "products")
    return df


def clean_sellers(df: pd.DataFrame) -> pd.DataFrame:
    """Sellers: same pattern as customers."""
    df = df.copy()
    df["seller_city"] = df["seller_city"].str.strip().str.lower()
    df["seller_state"] = df["seller_state"].str.strip().str.upper()
    validate_no_null_pk(df, "seller_id", "sellers")
    validate_no_duplicate_pk(df, ["seller_id"], "sellers")
    return df


# — Main Pipeline —————

def run_etl():
    log.info("Starting ETL pipeline...")
    engine = create_engine(DB_URL)

    # Load and clean each table
    tables = {
        "customers": (load_csv("olist_customers_dataset.csv"), clean_customers),
        "orders": (load_csv("olist_orders_dataset.csv"), clean_orders),
        "order_items": (load_csv("olist_order_items_dataset.csv"), clean_order_items)
    }

```



```

"order_payments": (load_csv("olist_order_payments_dataset.csv"), clean_order_payments),
"order_reviews": (load_csv("olist_order_reviews_dataset.csv"), clean_order_reviews),
"products": (load_csv("olist_products_dataset.csv"), clean_products),
"sellers": (load_csv("olist_sellers_dataset.csv"), clean_sellers),
"geolocation": (load_csv("olist_geolocation_dataset.csv"), lambda df: df),
"product_category_translation": (
    load_csv("product_category_name_translation.csv"), lambda df: df
),
}

# Load in dependency order (referenced tables before referencing)
load_order = [
    "customers", "products", "sellers", "geolocation",
    "product_category_translation", "orders", "order_items",
    "order_payments", "order_reviews"
]

for table_name in load_order:
    raw_df, clean_fn = tables[table_name]
    cleaned_df = clean_fn(raw_df)
    write_to_postgres(cleaned_df, table_name, engine)

log.info("ETL pipeline complete. All tables loaded successfully.")

if __name__ == "__main__":
    run_etl()

```

Phase 5 — Exploratory Data Analysis (EDA)

Analyst-Style EDA: The Right Mindset

Academic EDA says: "describe every variable." Analyst EDA says: "what business questions can this data answer, and what would a stakeholder actually want to know?"

Before writing a single line of code, write down 5–7 questions a real e-commerce team would ask:

1. How is our revenue trending month-over-month? (Revenue Trend)
2. Which states are our biggest markets? (Geographic Performance)
3. Which product categories drive the most revenue? (Category Mix)
4. Are customers buying more than once? (Retention Signal)

5. How often do we deliver on time, and how does that correlate with review scores? (Operational Quality)
6. What is the relationship between shipping delay and customer satisfaction? (Root Cause Analysis)
7. Who are our highest-value customers and where do they live? (Customer Value)

Every chart and SQL query in this project should answer one of these questions.

analysis/01_eda.ipynb — EDA Code

python

```
# =====
# OLIST EDA – Business-Oriented Exploratory Data Analysis
# Focus: answer business questions, not describe variables
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
import os
from sqlalchemy import create_engine
from dotenv import load_dotenv

load_dotenv()

engine = create_engine(
    f"postgresql+psycopg2://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}}"
    f"@{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB_NAME')}}"
)

# Style – clean and professional for portfolio
plt.rcParams.update({
    "figure.facecolor": "white",
    "axes.facecolor": "#f8f9fa",
    "axes.grid": True,
    "grid.alpha": 0.5,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.family": "sans-serif",
    "font.size": 11,
})
```

```

FIGURES_DIR = "outputs/figures"
os.makedirs(FIGURES_DIR, exist_ok=True)

def save_fig(name):
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, f"{name}.png"), dpi=150, bbox_inches="tight")
    plt.show()

# — 1. Revenue Trend (Monthly) —————
# Business question: How is revenue trending? Are we growing?
# Why this matters: The most fundamental KPI for any e-commerce business.

query_revenue = """
    SELECT
        DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
        SUM(oi.price + oi.freight_value)                AS total_revenue,
        COUNT(DISTINCT o.order_id)                      AS order_count,
        COUNT(DISTINCT o.customer_id)                  AS unique_customers
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 'delivered'
        AND o.order_purchase_timestamp >= '2017-01-01'
        AND o.order_purchase_timestamp < '2018-09-01' -- avoid incomplete months
    GROUP BY 1
    ORDER BY 1
"""

df_revenue = pd.read_sql(query_revenue, engine)
df_revenue["month"] = pd.to_datetime(df_revenue["month"])
df_revenue["revenue_growth_pct"] = df_revenue["total_revenue"].pct_change() * 100

fig, ax1 = plt.subplots(figsize=(12, 5))
ax2 = ax1.twinx()

ax1.bar(df_revenue["month"], df_revenue["total_revenue"] / 1e6,
        color="#4C72B0", alpha=0.8, width=25, label="Revenue (M BRL)")
ax2.plot(df_revenue["month"], df_revenue["order_count"],
        color="#DD8452", linewidth=2, marker="o", markersize=4, label="Order Count")

ax1.set_xlabel("Month")
ax1.set_ylabel("Revenue (Millions BRL)", color="#4C72B0")
ax2.set_ylabel("Order Count", color="#DD8452")
ax1.yaxis.set_major_formatter(mtick.FuncFormatter(lambda x, _: f"R${x:.1f}M"))
lines1, labels1 = ax1.get_legend_handles_labels()

```

```

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc="upper left")

plt.title("Monthly Revenue & Order Volume\nOlist E-Commerce Platform (2017-2018)",
save_fig("01_monthly_revenue_trend")

# — Insight for stakeholders: _____
# "Revenue grew approximately 20x from Jan 2017 to Aug 2018, with a notable
# acceleration in late 2017 Q4 (Black Friday effect). Order volume closely
# tracks revenue, suggesting average order value has been stable – a healthy
# sign that growth is volume-driven, not price inflation."

# — 2. Geographic Sales Distribution _____
# Business question: Where are our customers?
# Why this matters: Informs logistics investment, marketing spend, and
# regional expansion decisions.

query_geo = """
    SELECT
        c.customer_state,
        COUNT(DISTINCT o.order_id)           AS order_count,
        SUM(oi.price + oi.freight_value)      AS total_revenue,
        AVG(oi.price + oi.freight_value)     AS avg_order_value
    FROM orders o
    JOIN customers c  ON o.customer_id = c.customer_id
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY 1
    ORDER BY order_count DESC
    LIMIT 15
"""

df_geo = pd.read_sql(query_geo, engine)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Chart A: Order volume by state
df_geo.sort_values("order_count").plot.barh(
    x="customer_state", y="order_count",
    ax=axes[0], color="#4C72B0", legend=False
)
axes[0].set_title("Orders by Customer State (Top 15)")
axes[0].set_xlabel("Order Count")
axes[0].axis('tight')
axes[0].xaxis.set_major_formatter(mpl.ticker.FuncFormatter(lambda v, pos: f'{v:05}'))

```

```

axes[0].xaxis.set_major_formatter(mtick.FuncFormatter(lambda x, _ : f"{x:,.0f}"))

# Chart B: Avg order value by state
df_geo.sort_values("avg_order_value").plot.barh(
    x="customer_state", y="avg_order_value",
    ax=axes[1], color="#DD8452", legend=False
)
axes[1].set_title("Avg Order Value by State (Top 15 by volume)")
axes[1].set_xlabel("Avg Order Value (BRL)")
axes[1].xaxis.set_major_formatter(mtick.FuncFormatter(lambda x, _ : f"R${x:,.0f}"))

save_fig("02_geographic_distribution")

```

— 3. Product Category Performance

Business question: Which categories drive revenue? Which have poor reviews?
 # Why this matters: Category mix informs merchandising, catalog expansion,
 # and supplier negotiations.

```

query_categories = """
    SELECT
        COALESCE(t.product_category_name_english, p.product_category_name, 'Unknown') AS category,
        COUNT(DISTINCT o.order_id) AS order_count,
        SUM(oi.price) AS total_revenue,
        AVG(r.review_score) AS avg_review_score,
        COUNT(DISTINCT p.product_id) AS product_count
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    JOIN products p ON oi.product_id = p.product_id
    LEFT JOIN product_category_translation t ON p.product_category_name = t.product_category_name
    LEFT JOIN order_reviews r ON o.order_id = r.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY 1
    HAVING COUNT(DISTINCT o.order_id) >= 100 -- exclude long-tail categories
    ORDER BY total_revenue DESC
    LIMIT 20
"""

df_cat = pd.read_sql(query_categories, engine)

fig, ax = plt.subplots(figsize=(12, 7))
colors = plt.cm.RdYlGn(
    (df_cat["avg_review_score"] - df_cat["avg_review_score"].min()) /
    (df_cat["avg_review_score"].max() - df_cat["avg_review_score"].min())
)

```

```

bars = ax.barh(df_cat["category"], df_cat["total_revenue"] / 1e6, color=colors)
ax.set_xlabel("Total Revenue (Millions BRL)")
ax.set_title("Revenue by Product Category\n(Color = avg review score: red=low, green=high)")
ax.xaxis.set_major_formatter(mtick.FuncFormatter(lambda x, _: f"R${x:.1f}M"))

sm = plt.cm.ScalarMappable(cmap="RdYlGn", norm=plt.Normalize(
    df_cat["avg_review_score"].min(), df_cat["avg_review_score"].max()))
plt.colorbar(sm, ax=ax, label="Avg Review Score")

save_fig("03_category_performance")

```

— 4. Delivery Performance vs Review Score

Business question: Does late delivery hurt satisfaction?

Why this matters: Operational teams use this to justify investment in logistics. If late delivery tanks reviews, that's a business case for faster shipping.

```

query_delivery = """
    SELECT
        o.order_id,
        EXTRACT(DAY FROM (
            o.order_delivered_customer_date - o.order_estimated_delivery_date
        )) AS delivery_delay_days,
        r.review_score
    FROM orders o
    JOIN order_reviews r ON o.order_id = r.order_id
    WHERE o.order_status = 'delivered'
        AND o.order_delivered_customer_date IS NOT NULL
        AND o.order_estimated_delivery_date IS NOT NULL
    """

df_delivery = pd.read_sql(query_delivery, engine)

# Bin by delay category
df_delivery["delay_bucket"] = pd.cut(
    df_delivery["delivery_delay_days"],
    bins=[-999, -14, -7, -1, 0, 7, 14, 999],
    labels=["Very Early\n(>14d)", "Early\n(7-14d)", "Early\n(1-7d)",
            "On Time", "Late\n(1-7d)", "Late\n(7-14d)", "Very Late\n(>14d)"]
)

review_by_delay = (df_delivery.groupby("delay_bucket", observed=True)["review_score"]
                    .agg(["mean", "count"]).reset_index())

```

```

fig, ax = plt.subplots(figsize=(12, 5))
colors = ["#2ecc71" if str(b) in ["Very Early\n(>14d)", "Early\n(7-14d)", "Early\n(1-7d)"]
          else "#e74c3c" for b in review_by_delay["delay_bucket"]]

bars = ax.bar(review_by_delay["delay_bucket"].astype(str), review_by_delay["mean"],
              color=colors)
ax.axhline(y=df_delivery["review_score"].mean(), color="gray", linestyle="--",
           linewidth=1, label=f"Overall Avg: {df_delivery['review_score'].mean():.2f}")
ax.set_ylim(1, 5.2)
ax.set_ylabel("Average Review Score")
ax.set_title("Customer Review Score by Delivery Timing\nEarly delivery → better review")
ax.legend()

# Annotate with order counts
for bar, count in zip(bars, review_by_delay["count"]):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.05,
            f"n={count:,}", ha="center", va="bottom", fontsize=9)

save_fig("04_delivery_vs_reviews")

```

— 5. Payment Method Mix

```

query_payments = """
    SELECT
        payment_type,
        COUNT(*)           AS transaction_count,
        SUM(payment_value)  AS total_value,
        AVG(payment_installments) AS avg_installments
    FROM order_payments
    GROUP BY payment_type
    ORDER BY total_value DESC
"""

df_pay = pd.read_sql(query_payments, engine)
df_pay = df_pay[df_pay["payment_type"] != "not_defined"]

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].pie(df_pay["transaction_count"],
            labels=df_pay["payment_type"],
            autopct="%1.1f%%", startangle=140,
            colors=sns.color_palette("Set2"))
axes[0].set_title("Payment Method Share\n(by transaction count)")

axes[1].bar(df_pay["payment_type"], df_pay["avg_installments"], color=sns.color_palette("Set2"))

```

```

axes[1].bar(df_pay["payment_type"], df_pay["avg_installments"], color=sns.color_palette("magma", 2))
axes[1].set_title("Avg Installments by Payment Type\n(Brazil uses installment credit)")
axes[1].set_ylabel("Avg Installments")

save_fig("05_payment_mix")

print("EDA complete. Figures saved to outputs/figures/")

```


Phase 6 — SQL Analytics

How Analysts Think About SQL

SQL is not just a retrieval language — it's a thinking tool. Good analyst SQL follows this pattern:

1. **Start with the grain** — what does one row of my result represent?
2. **Work from the inside out** — start with the smallest unit (order_items), then aggregate up
3. **Use CTEs to make logic readable** — not subquery inception
4. **Name things clearly** — `total_revenue` not `sum1`

analysis/02_sql_analysis.sql

```

sql

-- =====
-- OLIST SQL ANALYTICS - 20 Business Queries
-- Organized from basic → advanced
-- =====

-- — SECTION 1: BASIC AGGREGATIONS —————

-- Query 1: Total delivered revenue
-- Grain: one row (overall summary)
-- Why: The single most important business number
SELECT
    COUNT(DISTINCT o.order_id)           AS total_orders,
    SUM(oi.price + oi.freight_value)     AS total_revenue,
    AVG(oi.price + oi.freight_value)     AS avg_order_value,
    COUNT(DISTINCT o.customer_id)       AS total_customers
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 'delivered';

```



```
WHERE o.order_status = 'delivered',
```

```
-- Query 2: Revenue by order status
```

```
-- Why: Shows what % of orders actually complete vs cancel/fail
```

```
SELECT
```

```
    order_status,
    COUNT(*)                               AS order_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER ()), 2) AS pct_of_total
```

```
FROM orders
```

```
GROUP BY order_status
```

```
ORDER BY order_count DESC;
```

```
-- Note: SUM(COUNT(*) OVER ()) is a window function – total orders across all status
```

```
-- Query 3: Monthly revenue trend
```

```
-- Grain: one row per month
```

```
-- Why: Revenue trend is the foundation of any business review
```

```
SELECT
```

```
    DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
    COUNT(DISTINCT o.order_id)                      AS order_count,
    ROUND(SUM(oi.price + oi.freight_value)::NUMERIC, 2) AS total_revenue,
    ROUND(AVG(oi.price + oi.freight_value)::NUMERIC, 2) AS avg_order_value
```

```
FROM orders o
```

```
JOIN order_items oi ON o.order_id = oi.order_id
```

```
WHERE o.order_status = 'delivered'
```

```
GROUP BY 1
```

```
ORDER BY 1;
```

```
-- — SECTION 2: JOINS AND DIMENSIONS
```

```
-- Query 4: Revenue by customer state
```

```
-- Grain: one row per state
```

```
-- New concept: multi-table JOIN – customer → order → items
```

```
SELECT
```

```
    c.customer_state,
    COUNT(DISTINCT o.order_id) AS order_count,
    ROUND(SUM(oi.price + oi.freight_value)::NUMERIC, 2) AS total_revenue,
    ROUND(AVG(oi.price + oi.freight_value)::NUMERIC, 2) AS avg_order_value
```

```
FROM customers c
```

```
JOIN orders o      ON c.customer_id = o.customer_id
```

```
JOIN order_items oi ON o.order_id = oi.order_id
```

```
WHERE o.order_status = 'delivered'
```

```
GROUP BY c.customer_state
```

```
ORDER BY total_revenue DESC;
```

```
-- Query 5: Revenue by product category (English names)
```

```
-- New concept: LEFT JOIN to translation table – products without translations still
```

```
SELECT
    COALESCE(t.product_category_name_english, p.product_category_name, 'Unknown') AS product_category_name,
    COUNT(DISTINCT oi.order_id) AS order_count,
    ROUND(SUM(oi.price)::NUMERIC, 2) AS product_revenue,
    ROUND(AVG(r.review_score)::NUMERIC, 2) AS avg_review
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
LEFT JOIN product_category_translation t
    ON p.product_category_name = t.product_category_name
LEFT JOIN order_reviews r ON oi.order_id = r.order_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status = 'delivered'
GROUP BY 1
HAVING COUNT(DISTINCT oi.order_id) >= 100
ORDER BY product_revenue DESC
LIMIT 20;
```

```
-- — SECTION 3: CASE WHEN
```

```
-- Query 6: Order volume by day of week
```

```
-- New concept: CASE WHEN for custom categorization
```

```
SELECT
    TO_CHAR(order_purchase_timestamp, 'Day') AS day_name,
    EXTRACT(DOW FROM order_purchase_timestamp) AS day_num,
    COUNT(*) AS order_count,
    CASE
        WHEN EXTRACT(DOW FROM order_purchase_timestamp) IN (0, 6) THEN 'Weekend'
        ELSE 'Weekday'
    END AS day_type
FROM orders
WHERE order_status = 'delivered'
GROUP BY 1, 2
ORDER BY day_num;
```

```
-- Query 7: Revenue and review buckets – classify order quality
```

```
-- CASE WHEN creates custom segments: useful for dashboards
```

```
SELECT
    CASE
```

```

        WHEN r.review_score = 5 THEN '5 - Excellent'
        WHEN r.review_score = 4 THEN '4 - Good'
        WHEN r.review_score = 3 THEN '3 - Neutral'
        WHEN r.review_score IN (1, 2) THEN '1-2 - Poor'
        ELSE 'No Review'
    END AS review_tier,
    COUNT(DISTINCT o.order_id) AS order_count,
    ROUND(SUM(oi.price + oi.freight_value)::NUMERIC, 2) AS total_revenue,
    ROUND(AVG(oi.price + oi.freight_value)::NUMERIC, 2) AS avg_order_value
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
LEFT JOIN order_reviews r ON o.order_id = r.order_id
WHERE o.order_status = 'delivered'
GROUP BY 1
ORDER BY review_tier;

```

-- — SECTION 4: CTEs —————

-- CTEs (WITH clauses) make complex logic readable by naming intermediate steps.

-- Think of them as named temporary tables within your query.

-- Query 8: Top-10 customers by lifetime value

-- CTE: calculate lifetime value first, then rank

```

WITH customer_ltv AS (
    SELECT
        c.customer_unique_id, -- IMPORTANT: use unique_id, not customer_id
        COUNT(DISTINCT o.order_id) AS lifetime_orders,
        ROUND(SUM(oi.price + oi.freight_value)::NUMERIC, 2) AS lifetime_revenue,
        MIN(o.order_purchase_timestamp) AS first_order_date,
        MAX(o.order_purchase_timestamp) AS last_order_date
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY c.customer_unique_id
)
SELECT *
FROM customer_ltv
ORDER BY lifetime_revenue DESC
LIMIT 10;

```

-- Query 9: Delivery performance summary

-- CTE: classify every order as early/on-time/late

```

WITH delivery_performance AS (

```

```

SELECT
    order_id,
    order_delivered_customer_date,
    order_estimated_delivery_date,
    EXTRACT(DAY FROM (
        order_delivered_customer_date - order_estimated_delivery_date
    )) AS delay_days,
    CASE
        WHEN order_delivered_customer_date <= order_estimated_delivery_date THEN
        WHEN order_delivered_customer_date > order_estimated_delivery_date
            AND EXTRACT(DAY FROM (order_delivered_customer_date - order_estimated_delivery_date)) < 8
        THEN 'Slightly Late (1-7d)'
        ELSE 'Very Late (>7d)'
    END AS delivery_tier
FROM orders
WHERE order_status = 'delivered'
    AND order_delivered_customer_date IS NOT NULL
    AND order_estimated_delivery_date IS NOT NULL
)
SELECT
    delivery_tier,
    COUNT(*) AS order_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS pct_of_delivered,
    ROUND(AVG(delay_days)::NUMERIC, 1) AS avg_delay_days
FROM delivery_performance
GROUP BY delivery_tier
ORDER BY order_count DESC;

```

-- Query 10: Monthly growth rate using LAG window function

-- Why: Percentage change requires comparing current vs previous row → window function

```

WITH monthly_rev AS (
    SELECT
        DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
        SUM(oi.price + oi.freight_value) AS revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY 1
)
SELECT
    month,
    ROUND(revenue::NUMERIC, 2) AS revenue,
    LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
    ROUND(
        (revenue - prev_month_revenue) / prev_month_revenue * 100, 2
    ) AS growth_rate
FROM monthly_rev

```

```

        (revenue - LAG(revenue) OVER (ORDER BY month)) /
        NULLIF(LAG(revenue) OVER (ORDER BY month), 0) * 100
    , 1)
    AS mom_growth_pct
FROM monthly_rev
ORDER BY month;

```

-- — SECTION 5: WINDOW FUNCTIONS —————

-- Window functions compute values across a set of rows related to the current row.
 -- They DO NOT collapse rows (unlike GROUP BY). This is the key difference.

-- Query 11: Rank sellers by revenue within each state

-- ROW_NUMBER() / RANK() – rank within a partition

```

SELECT
    s.seller_state,
    s.seller_id,
    ROUND(SUM(oi.price)::NUMERIC, 2)    AS seller_revenue,
    RANK() OVER (
        PARTITION BY s.seller_state
        ORDER BY SUM(oi.price) DESC
    )
    AS state_rank
FROM sellers s
JOIN order_items oi ON s.seller_id = oi.seller_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status = 'delivered'
GROUP BY s.seller_state, s.seller_id
QUALIFY state_rank <= 3 -- top 3 sellers per state (PostgreSQL: use subquery instead)
ORDER BY s.seller_state, state_rank;

```

-- PostgreSQL version (no QUALIFY):

-- Wrap in a subquery and filter state_rank <= 3

-- Query 12: Running total revenue

```

SELECT
    DATE_TRUNC('month', o.order_purchase_timestamp)    AS month,
    SUM(oi.price + oi.freight_value)                  AS monthly_revenue,
    SUM(SUM(oi.price + oi.freight_value)) OVER (
        ORDER BY DATE_TRUNC('month', o.order_purchase_timestamp)
    )
    AS cumulative_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 'delivered'
GROUP BY 1
ORDER BY 1;

```

```
-- Note: SUM(SUM(...)) OVER (): the inner SUM is the GROUP BY aggregation;
-- the outer SUM OVER is the window function running over those aggregate values.

-- Query 13: Revenue percentile for each category
-- NTILE / PERCENT_RANK: where does a category fall in the distribution?
WITH category_revenue AS (
    SELECT
        COALESCE(t.product_category_name_english, p.product_category_name) AS category,
        SUM(oi.price) AS revenue
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    LEFT JOIN product_category_translation t ON p.product_category_name = t.product_category_name
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY 1
)
SELECT
    category,
    ROUND(revenue::NUMERIC, 2) AS revenue,
    NTILE(4) OVER (ORDER BY revenue) AS revenue_quartile,
    ROUND(
        PERCENT_RANK() OVER (ORDER BY revenue) * 100
        , 1) AS revenue_percentile
FROM category_revenue
ORDER BY revenue DESC;
```

-- — SECTION 6: ADVANCED BUSINESS QUERIES —————

```
-- Query 14: Repeat purchase rate
-- Why: Retention is the most important metric for subscription/e-commerce businesses
-- A repeat rate of >10% is good for marketplace e-commerce.
```

```
WITH customer_order_counts AS (
    SELECT
        c.customer_unique_id,
        COUNT(DISTINCT o.order_id) AS total_orders
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.order_status = 'delivered'
    GROUP BY c.customer_unique_id
)
SELECT
    COUNT(*) AS total_unique_customers,
    SUM(CASE WHEN total_orders >= 2 THEN 1 ELSE 0 END) AS repeat_customers,
```

```

ROUND(
    SUM(CASE WHEN total_orders >= 2 THEN 1 ELSE 0 END) * 100.0 / COUNT(*)
    , 2)
    AS repeat_rate_pct
FROM customer_order_counts;

```

-- Query 15: Revenue contribution of top 20% customers (Pareto principle)
 -- This is a classic interview question: "Do 20% of customers drive 80% of revenue?"

```

WITH customer_revenue AS (
    SELECT
        c.customer_unique_id,
        SUM(oi.price + oi.freight_value)    AS lifetime_revenue
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY c.customer_unique_id
),
ranked AS (
    SELECT *,
        NTILE(5) OVER (ORDER BY lifetime_revenue DESC) AS revenue_quintile
    FROM customer_revenue
)
SELECT
    revenue_quintile,
    COUNT(*)
    AS customer_count,
    ROUND(SUM(lifetime_revenue)::NUMERIC, 2)
    AS total_revenue,
    ROUND(
        SUM(lifetime_revenue) * 100.0 / SUM(SUM(lifetime_revenue)) OVER ()
        , 2)
    AS pct_of_total_revenue
FROM ranked
GROUP BY revenue_quintile
ORDER BY revenue_quintile;

```

-- Query 16: Average days between purchase and delivery, by state
 -- Business question: Are some states getting worse service?

```

SELECT
    c.customer_state,
    COUNT(o.order_id)    AS delivered_orders,
    ROUND(AVG(
        EXTRACT(DAY FROM (
            o.order_delivered_customer_date - o.order_purchase_timestamp
        ))
    )::NUMERIC, 1)
    AS avg_delivery_days,

```

```

ROUND(AVG(r.review_score)::NUMERIC, 2) AS avg_review_score
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
LEFT JOIN order_reviews r ON o.order_id = r.order_id
WHERE o.order_status = 'delivered'
      AND o.order_delivered_customer_date IS NOT NULL
GROUP BY c.customer_state
ORDER BY avg_delivery_days DESC;

```

-- Query 17: First-time vs repeat customer revenue

-- Business question: How much revenue comes from new vs returning customers?

```

WITH customer_cohort AS (
    SELECT
        c.customer_unique_id,
        o.order_id,
        oi.price + oi.freight_value AS order_value,
        ROW_NUMBER() OVER (
            PARTITION BY c.customer_unique_id
            ORDER BY o.order_purchase_timestamp
        ) AS order_num
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 'delivered'
)
SELECT
    CASE WHEN order_num = 1 THEN 'First Purchase' ELSE 'Repeat Purchase' END AS customer_type,
    COUNT(DISTINCT order_id) AS order_count,
    ROUND(SUM(order_value)::NUMERIC, 2) AS total_revenue,
    ROUND(AVG(order_value)::NUMERIC, 2) AS avg_order_value
FROM customer_cohort
GROUP BY 1;

```

-- Query 18: Seller performance scorecard

-- Multiple KPIs per seller in one query – a common dashboard data source

```

WITH seller_metrics AS (
    SELECT
        s.seller_id,
        s.seller_state,
        COUNT(DISTINCT oi.order_id) AS orders_fulfilled,
        ROUND(SUM(oi.price)::NUMERIC, 2) AS gross_revenue,
        ROUND(AVG(r.review_score)::NUMERIC, 2) AS avg_review,
        ROUND(AVG(

```



```

        EXTRACT(DAY FROM (
            o.order_delivered_customer_date - o.order_purchase_timestamp
        ))
    )::NUMERIC, 1) AS avg_delivery_days
FROM sellers s
JOIN order_items oi ON s.seller_id = oi.seller_id
JOIN orders o ON oi.order_id = o.order_id
LEFT JOIN order_reviews r ON o.order_id = r.order_id
WHERE o.order_status = 'delivered'
    AND o.order_delivered_customer_date IS NOT NULL
GROUP BY s.seller_id, s.seller_state
HAVING COUNT(DISTINCT oi.order_id) >= 20 -- exclude micro-sellers
)
SELECT
    *,
    RANK() OVER (ORDER BY gross_revenue DESC) AS revenue_rank,
    RANK() OVER (ORDER BY avg_review DESC) AS review_rank
FROM seller_metrics
ORDER BY gross_revenue DESC
LIMIT 20;

```

-- Query 19: Payment installment analysis by product category
 -- Brazil heavily uses installment credit – this surfaces category-level patterns

```

SELECT
    COALESCE(t.product_category_name_english, p.product_category_name) AS category,
    ROUND(AVG(op.payment_installments)::NUMERIC, 2) AS avg_installments,
    ROUND(AVG(oi.price)::NUMERIC, 2) AS avg_item_price,
    COUNT(DISTINCT oi.order_id) AS order_count
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
LEFT JOIN product_category_translation t ON p.product_category_name = t.product_cat
JOIN order_payments op ON oi.order_id = op.order_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status = 'delivered'
    AND op.payment_type = 'credit_card'
GROUP BY 1
HAVING COUNT(DISTINCT oi.order_id) >= 100
ORDER BY avg_installments DESC
LIMIT 15;

```

-- Query 20: Monthly cohort first-purchase counts
 -- This feeds directly into the cohort retention heatmap in Phase 7

```

SELECT

```

```

DATE_TRUNC('month', MIN(o.order_purchase_timestamp)) AS cohort_month,
COUNT(DISTINCT c.customer_unique_id) AS cohort_size
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_status = 'delivered'
GROUP BY c.customer_unique_id
HAVING DATE_TRUNC('month', MIN(o.order_purchase_timestamp)) BETWEEN '2017-01-01' AND
-- GROUP BY cohort_month:
SELECT cohort_month, COUNT(*) AS cohort_size
FROM (
    SELECT
        c.customer_unique_id,
        DATE_TRUNC('month', MIN(o.order_purchase_timestamp)) AS cohort_month
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.order_status = 'delivered'
    GROUP BY c.customer_unique_id
) sub
GROUP BY cohort_month
ORDER BY cohort_month;

```


Phase 7 — Cohort Retention Analysis

What Is Cohort Analysis and Why Does It Matter?

A cohort is a group of customers who share a common characteristic defined by a time window — typically their first purchase month. Cohort retention analysis answers: *of all customers who first bought in month X, what fraction came back in subsequent months?*

Why business teams care:

- Product teams use it to measure whether product improvements actually increase retention
- Marketing teams use it to evaluate CAC payback periods (how fast does a new customer pay back their acquisition cost?)
- Finance teams use it to model LTV (lifetime value)

A cohort retention heatmap is one of the most immediately impressive portfolio visualizations you can produce — it looks sophisticated and it communicates something meaningful.

analysis/03_cohort_retention.ipynb

python

```
# =====  
# OLIST COHORT RETENTION ANALYSIS  
# Approach: SQL extracts the raw data; pandas builds the matrix  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
import os  
from sqlalchemy import create_engine  
from dotenv import load_dotenv  
  
load_dotenv()  
engine = create_engine(  
    f"postgresql+psycopg2://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}}"  
    f"@{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB_NAME')}}"  
)
```

— Step 1: Extract Customer-Level Purchase History —————

This query gives us every customer's purchase dates.

We'll compute cohort membership and period offsets in Python.

```
query = """  
    SELECT  
        c.customer_unique_id,  
        DATE_TRUNC('month', o.order_purchase_timestamp) AS order_month  
    FROM customers c  
    JOIN orders o ON c.customer_id = o.customer_id  
    WHERE o.order_status = 'delivered'  
        AND o.order_purchase_timestamp BETWEEN '2017-01-01' AND '2018-08-31'  
    ORDER BY c.customer_unique_id, order_month  
    """
```

```
df = pd.read_sql(query, engine)  
df["order_month"] = pd.to_datetime(df["order_month"])
```

— Step 2: Assign Cohort (First Purchase Month) —————

Each customer's cohort is their FIRST order month.

```
cohort_map = (  
    df.groupby("customer_unique_id")["order_month"]  
    .first()
```

```

    .min()
    .rename("cohort_month")
    .reset_index()
)

df = df.merge(cohort_map, on="customer_unique_id", how="left")

# — Step 3: Compute Period Offset —————
# Period 0 = cohort month (first purchase)
# Period 1 = one month after first purchase
# etc.

df["period"] = (
    (df["order_month"].dt.year - df["cohort_month"].dt.year) * 12
    + (df["order_month"].dt.month - df["cohort_month"].dt.month)
)

# — Step 4: Build Cohort Retention Table —————
# Count unique customers per (cohort_month, period)

cohort_data = (
    df.groupby(["cohort_month", "period"])["customer_unique_id"]
    .nunique()
    .reset_index()
    .rename(columns={"customer_unique_id": "customers"})
)

# Pivot: rows = cohort month, columns = period
retention_table = cohort_data.pivot_table(
    index="cohort_month", columns="period", values="customers"
)

# Cohort sizes = period 0 column
cohort_sizes = retention_table[0]

# Convert to retention rate: divide each row by its cohort size
retention_rate = retention_table.divide(cohort_sizes, axis=0) * 100

# Focus on cohorts with full 12-month data (avoid truncation artifacts)
retention_rate = retention_rate.loc[
    retention_rate.index <= "2017-09-01",
    range(0, 12)
]

# — Step 5: Visualize – Heatmap —————

```

```

fig, ax = plt.subplots(figsize=(14, 8))

# Format cohort labels
retention_rate.index = pd.to_datetime(retention_rate.index).strftime("%b %Y")
retention_rate.columns = [f"Month {i}" for i in retention_rate.columns]

mask = retention_rate.isnull()

sns.heatmap(
    retention_rate,
    mask=mask,
    annot=True,
    fmt=".1f",
    cmap="YlOrRd_r",          # green = high retention, red = low
    vmin=0, vmax=100,
    linewidths=0.5,
    linecolor="white",
    ax=ax,
    annot_kws={"size": 9},
    cbar_kws={"label": "Retention Rate (%)"}
)

ax.set_title(
    "Monthly Cohort Retention Heatmap\n"
    "% of customers who purchased in month N who returned in month N+X",
    pad=20, fontsize=14
)

ax.set_xlabel("Month Since First Purchase")
ax.set_ylabel("Acquisition Cohort")
plt.xticks(rotation=0)
plt.yticks(rotation=0)

plt.tight_layout()
plt.savefig("outputs/figures/07_cohort_retention_heatmap.png", dpi=150, bbox_inches="tight")
plt.show()

# — Step 6: Print Key Insights —————

m1_retention = retention_rate["Month 1"].dropna().mean()
m3_retention = retention_rate["Month 3"].dropna().mean()
m6_retention = retention_rate["Month 6"].dropna().mean()

print(f"Average Month-1 Retention: {m1_retention:.1f}%")
print(f"Average Month-3 Retention: {m3_retention:.1f}%")

```

```
print(f"Average Month-6 Retention: {m6_retention:.1f}%")
print()
print("Insight: Olist is a marketplace (like Amazon), not a subscription product.")
print("Very low retention (~3-5%) is expected and normal for this business model.")
print("The insight is not 'retention is bad' – it's 'marketplace customers don't ret")
print("A follow-up analysis could compare high-retention categories vs low-retention")
```

How to Communicate This to a Stakeholder

When presenting cohort analysis, never just show the heatmap. Provide:

1. **Context:** "Month 0 is always 100% because that's the acquisition period."
2. **Benchmark:** "For a marketplace model, 3-5% month-1 retention is industry-typical. Subscription products target 60-80%."
3. **Actionable insight:** "The 2017-Q4 cohorts show slightly higher retention — this aligns with holiday season shoppers who may have received gift cards and returned."

Phase 8 — RFM Customer Segmentation

What Is RFM and Why Do Businesses Use It?

RFM stands for:

- **Recency:** How recently did a customer last purchase? (days since last order)
- **Frequency:** How many times have they purchased? (order count)
- **Monetary:** How much have they spent? (total revenue)

RFM is a classic marketing analytics framework used by CRM teams, email marketing platforms, and growth teams to segment customers into actionable buckets. A marketing team might run a "win-back" campaign for At-Risk customers, a loyalty program for Champions, and an upsell campaign for Potential Loyalists.

The key value for your portfolio: RFM demonstrates you can translate raw data into **actionable business segments**, not just metrics.

analysis/04_rfm_segmentation.ipynb

```
python
# =====
# OLIST RFM CUSTOMER SEGMENTATION
```

```
# =====

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import os
from sqlalchemy import create_engine
from dotenv import load_dotenv

load_dotenv()
engine = create_engine(
    f"postgresql+psycopg2://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}}"
    f"@{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB_NAME')}}"
)
```

— Step 1: Compute Raw RFM Metrics

```
ANALYSIS_DATE = pd.Timestamp("2018-10-01") # Day after dataset ends

query = """
    SELECT
        c.customer_unique_id,
        MAX(o.order_purchase_timestamp)           AS last_purchase_date,
        COUNT(DISTINCT o.order_id)                 AS frequency,
        SUM(oi.price + oi.freight_value)           AS monetary
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY c.customer_unique_id
    """

df_rfm = pd.read_sql(query, engine)
df_rfm["last_purchase_date"] = pd.to_datetime(df_rfm["last_purchase_date"])
df_rfm["recency"] = (ANALYSIS_DATE - df_rfm["last_purchase_date"]).dt.days

print(f"Total customers: {len(df_rfm):,}")
print(df_rfm[["recency", "frequency", "monetary"]].describe().round(2))
```

— Step 2: Assign RFM Scores (1-5 scale)

```
# We use quintile scoring: the top 20% on recency gets a score of 5, etc.
# IMPORTANT: Recency is inverted – lower days = more recent = higher score
```

```
def rfm_score(df, col, ascending=True, n=5):
    """Assign quintile scores. ascending=True means higher value = higher score."""
    labels = list(range(1, n + 1)) if ascending else list(range(n, 0, -1))
    return pd.qcut(df[col], q=n, labels=labels, duplicates="drop")

df_rfm["R_score"] = rfm_score(df_rfm, "recency", ascending=False) # lower recency =
df_rfm["F_score"] = rfm_score(df_rfm, "frequency", ascending=True)
df_rfm["M_score"] = rfm_score(df_rfm, "monetary", ascending=True)

df_rfm["R_score"] = df_rfm["R_score"].astype(int)
df_rfm["F_score"] = df_rfm["F_score"].astype(int)
df_rfm["M_score"] = df_rfm["M_score"].astype(int)

df_rfm["rfm_score"] = (
    df_rfm["R_score"].astype(str)
    + df_rfm["F_score"].astype(str)
    + df_rfm["M_score"].astype(str)
)

df_rfm["rfm_combined"] = df_rfm["R_score"] + df_rfm["F_score"] + df_rfm["M_score"]
```

— Step 3: Assign Business Segments

These segment rules come from standard marketing analytics practice.
 # The names are memorable and immediately actionable for a marketing team.

```
def assign_segment(row):
    r, f, m = row["R_score"], row["F_score"], row["M_score"]
    rfm = row["rfm_combined"]

    if r >= 4 and f >= 4 and m >= 4:
        return "Champions"
    elif r >= 3 and f >= 3:
        return "Loyal Customers"
    elif r >= 4 and f <= 2:
        return "Recent Customers"
    elif r >= 3 and f >= 1 and m >= 2:
        return "Potential Loyalists"
    elif r == 3 and f == 1 and m == 1:
        return "Promising"
    elif r <= 2 and f >= 3:
        return "At Risk"
    elif r <= 2 and f <= 2 and m >= 3:
        return "Can't Lose Them"
```



```

elif r == 1 and f == 1:
    return "Lost"
else:
    return "Needs Attention"

df_rfm["segment"] = df_rfm.apply(assign_segment, axis=1)

```

— Step 4: Segment Summary Statistics

```

segment_summary = (
    df_rfm.groupby("segment")
    .agg(
        customer_count=("customer_unique_id", "count"),
        avg_recency=("recency", "mean"),
        avg_frequency=("frequency", "mean"),
        avg_monetary=("monetary", "mean"),
        total_revenue=("monetary", "sum")
    )
    .round(2)
    .sort_values("total_revenue", ascending=False)
)

print("\nRFM Segment Summary:")
print(segment_summary.to_string())

```

— Step 5: Visualizations

```

SEGMENT_COLORS = {
    "Champions": "#2ecc71",
    "Loyal Customers": "#27ae60",
    "Potential Loyalists": "#f39c12",
    "Recent Customers": "#3498db",
    "Promising": "#9b59b6",
    "Needs Attention": "#e67e22",
    "At Risk": "#e74c3c",
    "Can't Lose Them": "#c0392b",
    "Lost": "#95a5a6",
}

```

Chart 1: Customer count per segment

```

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

```

```

segment_summary_sorted = segment_summary.sort_values("customer_count", ascending=True)

```

```

colors = [SEGMENT_COLORS.get(s, "#aaa") for s in segment_summary_sorted.index]

axes[0].barh(
    segment_summary_sorted.index,
    segment_summary_sorted["customer_count"],
    color=colors
)
axes[0].set_title("Customers per Segment")
axes[0].set_xlabel("Customer Count")

# Chart 2: Revenue per segment
rev_sorted = segment_summary.sort_values("total_revenue", ascending=True)
colors2 = [SEGMENT_COLORS.get(s, "#aaa") for s in rev_sorted.index]

axes[1].barh(rev_sorted.index, rev_sorted["total_revenue"] / 1e6, color=colors2)
axes[1].set_title("Total Revenue per Segment (Millions BRL)")
axes[1].set_xlabel("Revenue (M BRL)")

plt.suptitle("RFM Customer Segmentation – Olist E-Commerce", fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig("outputs/figures/08a_rfm_segment_distribution.png", dpi=150, bbox_inches="tight")
plt.show()

# Chart 3: RFM scatter plot – Recency vs Monetary, colored by segment
fig, ax = plt.subplots(figsize=(12, 8))

for segment, group in df_rfm.groupby("segment"):
    ax.scatter(
        group["recency"],
        group["monetary"],
        alpha=0.3, s=15,
        color=SEGMENT_COLORS.get(segment, "#aaa"),
        label=segment
    )

ax.set_xlabel("Recency (Days Since Last Purchase)")
ax.set_ylabel("Monetary Value (BRL)")
ax.set_title("Customer Segments – Recency vs Monetary Value\n(RFM Segmentation)")
ax.set_xlim(0, df_rfm["recency"].quantile(0.99))
ax.set_ylim(0, df_rfm["monetary"].quantile(0.99))
ax.legend(bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.savefig("outputs/figures/08b_rfm_scatter.png", dpi=150, bbox_inches="tight")

```

```
plt.show()

# — Step 6: Export for Dashboard

os.makedirs("outputs/exports", exist_ok=True)
df_rfm[["customer_unique_id", "recency", "frequency", "monetary",
        "R_score", "F_score", "M_score", "rfm_combined", "segment"]].to_csv(
    "outputs/exports/rfm_segments.csv", index=False
)
segment_summary.to_csv("outputs/exports/rfm_segment_summary.csv")
print("RFM exports saved to outputs/exports/")
```

How Marketing/CRM Teams Use This

Segment	CRM Action
Champions	Reward program, early access, brand advocates
At Risk	Win-back email campaign: "We miss you — here's 15% off"
Potential Loyalists	Targeted upsell for complementary categories
Lost	Low-cost batch reactivation campaign or remove from list
Can't Lose Them	Personal outreach — these are high-value churning customers

Phase 9 — KPI Design

KPIs: The Difference Between Data and Decision Support

A KPI (Key Performance Indicator) is not just a metric. It's a metric that is: (1) tied to a business objective, (2) owned by a team, (3) reviewed on a cadence, and (4) compared against a target.

For your dashboard, define KPIs in three tiers:

Tier 1 — Executive KPIs (board/CEO level)

KPI	Formula	Business Decision
Total Revenue (MTD/YTD)	SUM(price + freight_value)	Overall growth trajectory

Monthly Revenue Growth Rate	$\frac{(\text{current} - \text{prior})}{\text{prior}}$	Acceleration vs deceleration
Total Active Customers	$\frac{\text{COUNT}(\text{DISTINCT customer_unique_id})}{\text{last 90 days}}$	Market penetration
Average Order Value (AOV)	$\frac{\text{Total Revenue}}{\text{Order Count}}$	Monetization efficiency

Tier 2 — Operational KPIs (ops/product)

KPI	Formula	Business Decision
On-Time Delivery Rate	$\frac{\% \text{ orders delivered on or before estimated date}}$	Logistics investment
Avg Delivery Days	$\text{AVG}(\text{delivered_date} - \text{purchase_date})$	Carrier performance
Avg Review Score	$\text{AVG}(\text{review_score})$	Product/service quality
% 1-2 Star Reviews	$\% \text{ orders with review} \leq 2$	Customer issue rate

Tier 3 — Growth KPIs (marketing/retention)

KPI	Formula	Business Decision
Repeat Purchase Rate	$\% \text{ customers with } \geq 2 \text{ orders}$	Retention investment
Customer LTV (estimated)	$\text{AOV} \times \text{Avg Frequency} \times \text{Avg Customer Lifespan}$	CAC budget
Champion Segment Size	$\% \text{ customers in Champion RFM tier}$	Brand loyalty
At-Risk Segment Size	$\% \text{ customers in At Risk tier}$	Win-back budget

Phase 10 — Dashboard Planning

Dashboard Storytelling Principles

Before building anything in Power BI or Tableau, sketch your dashboard layout. Real analysts do this on paper or in Figma before touching the tool.

The golden rule: one dashboard tells one story for one audience.

For this project, build two views:

View 1 — Executive Overview (audience: VP of E-Commerce, CFO)

- 4 KPI cards at top: Revenue (MTD), Orders (MTD), AOV, Repeat Rate
- Revenue trend line (18 months)
- Revenue by State map
- Top 5 categories by revenue (bar chart)

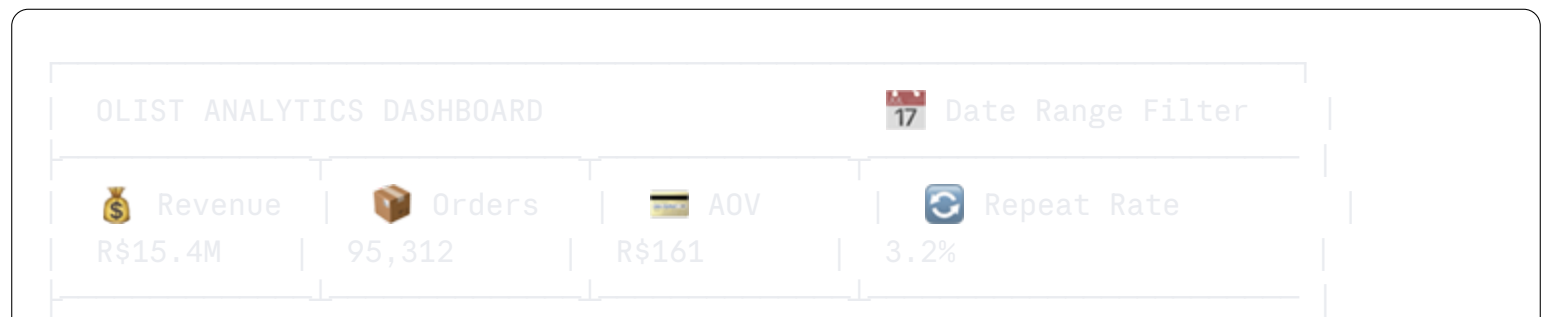
View 2 — Customer Intelligence (audience: Marketing, CRM team)

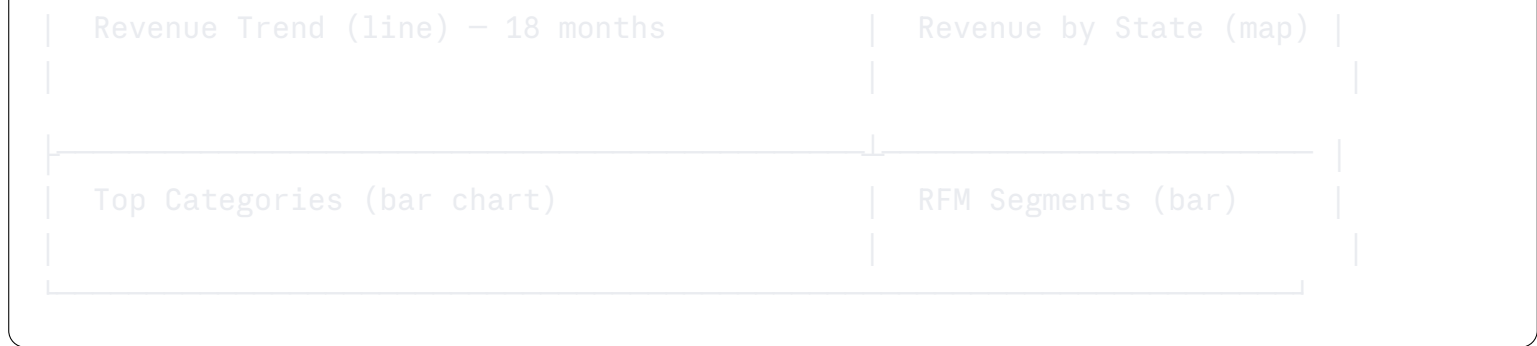
- RFM segment distribution (bar chart)
- Cohort retention heatmap
- Segment revenue contribution (stacked bar)
- Geographic customer density

Common Mistakes Students Make in Dashboards

1. **Too many charts on one page.** Recruiters scan dashboards in 15 seconds. 4-6 well-chosen charts beat 15 mediocre ones.
2. **No titles that explain the insight.** "Revenue by Month" is a label. "Revenue grew 20x over 18 months" is an insight.
3. **Inconsistent color schemes.** Pick 2-3 colors and use them consistently across all charts.
4. **No filters/slicers.** A dashboard without interactive filters looks like a static slide. Add date range, state, and category slicers.
5. **No summary text.** Real dashboards have a "Key Insights" text box that explains the 3 most important things a stakeholder should notice.

Layout Template





Phase 11 — Power BI Dashboard Build

Data Preparation: Export Analysis-Ready Tables

Before opening Power BI, export these tables from PostgreSQL:

```
python

# etl/export_for_dashboard.py
import pandas as pd
from sqlalchemy import create_engine
from dotenv import load_dotenv
import os

load_dotenv()
engine = create_engine(
    f"postgresql+psycopg2://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}@"
    f"{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB_NAME')}"
)

EXPORT_DIR = "outputs/exports"
os.makedirs(EXPORT_DIR, exist_ok=True)

# Fact table: orders with all dimensions joined
orders_fact = pd.read_sql("""
    SELECT
        o.order_id,
        o.order_purchase_timestamp,
        DATE_TRUNC('month', o.order_purchase_timestamp) AS order_month,
        o.order_status,
        c.customer_unique_id,
        c.customer_state,
        c.customer_city,
        oi.product_id,
```

```

COALESCE(t.product_category_name_english, p.product_category_name) AS category_name,
s.seller_id,
s.seller_state,

oi.price,
oi.freight_value,
oi.price + oi.freight_value AS total_value,
r.review_score,
EXTRACT(DAY FROM (o.order_delivered_customer_date - o.order_purchase_timestamp)) AS delivery_delay_in_days,
EXTRACT(DAY FROM (o.order_delivered_customer_date - o.order_estimated_delivery_date)) AS delivery_status_delay_in_days,
CASE WHEN o.order_delivered_customer_date <= o.order_estimated_delivery_date
      THEN 'On Time' ELSE 'Late' END AS delivery_status
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
LEFT JOIN product_category_translation t ON p.product_category_name = t.product_category_name
JOIN sellers s ON oi.seller_id = s.seller_id
LEFT JOIN order_reviews r ON o.order_id = r.order_id
WHERE o.order_status = 'delivered'
"""
engine)

orders_fact.to_csv(f"{EXPORT_DIR}/orders_fact.csv", index=False)
print(f"Exported orders_fact: {len(orders_fact):,} rows")

# RFM segments (from Phase 8)
# (already exported in Phase 8 to rfm_segments.csv)

print("Dashboard exports complete.")

```

Power BI Step-by-Step Setup

Step 1: Import Data

1. Open Power BI Desktop → Get Data → Text/CSV
2. Import `orders_fact.csv` and `rfm_segments.csv`
3. In the Query Editor: verify date columns are recognized as `Date/Time` type, not text

Step 2: Create a Date Table (best practice)

In Power BI, always create a separate Date dimension table. This enables time-intelligence functions.

In Power Query → New Query → Blank Query → Advanced Editor:

```

let
    StartDate = #date(2017, 1, 1),
    EndDate = #date(2018, 12, 31),
    DayCount = Duration.Days(EndDate - StartDate) + 1,
    Source = List.Dates(StartDate, DayCount, #duration(1, 0, 0, 0)),
    DateTable = Table.FromList(Source, Splitter.SplitByNothing(), {"Date"}),
    DateType = Table.TransformColumnTypes(DateTable, {"Date", type date}),
    Year = Table.AddColumn(DateType, "Year", each Date.Year([Date]), Int64.Type),
    Month = Table.AddColumn(Year, "Month", each Date.Month([Date]), Int64.Type),
    MonthName = Table.AddColumn(Month, "MonthName", each Date.ToText([Date], "MMM")),
    Quarter = Table.AddColumn(MonthName, "Quarter", each "Q" & Text.From(Date.Quarter([Date])), Int64.Type),
    YearMonth = Table.AddColumn(Quarter, "YearMonth", each Date.ToText([Date], "yyyy-MM")),
in
    YearMonth

```

Step 3: Create Relationships

In Model view:

- `orders_fact[order_purchase_timestamp (as date)]` → `DateTable[Date]` (many-to-one)
- `orders_fact[customer_unique_id]` → `rfm_segments[customer_unique_id]` (many-to-one)

Step 4: Create DAX Measures

DAX (Data Analysis Expressions) is Power BI's formula language. Create these in a dedicated "Measures" table.

dax


```

-- Create a blank table for measures: Modeling → New Table → Measures = ROW("placeholder")

Total Revenue = SUM(orders_fact[total_value])

Total Orders = DISTINCTCOUNT(orders_fact[order_id])

AOV = DIVIDE([Total Revenue], [Total Orders])

Total Customers = DISTINCTCOUNT(orders_fact[customer_unique_id])

Avg Review Score = AVERAGE(orders_fact[review_score])

On Time Delivery Rate % =
    DIVIDE(
        CALCULATE(COUNTROWS(orders_fact), orders_fact[delivery_status] = "On Time"),
        COUNTROWS(orders_fact)
    ) * 100

Revenue MoM Growth % =
    VAR CurrentRevenue = [Total Revenue]
    VAR PriorRevenue = CALCULATE(
        [Total Revenue],
        DATEADD(DateTable[Date], -1, MONTH)
    )
    RETURN DIVIDE(CurrentRevenue - PriorRevenue, PriorRevenue) * 100

```

Step 5: Build the Dashboard

KPI Cards (top row):

- Insert → Card visual
- Field: `[Total Revenue]`, format as currency
- Repeat for Orders, AOV, On-Time Delivery Rate
- Use conditional formatting on the MoM growth card: green if positive, red if negative

Revenue Trend (Line Chart):

- X-axis: `DateTable[YearMonth]`
- Y-axis: `[Total Revenue]`
- Add a trend line (Analytics pane → Trend Line)
- Title: "Revenue Trend — 2017 to 2018"

Revenue by State (Filled Map):

- Location: `orders_fact[customer_state]`
- Values: `[Total Revenue]`
- Color saturation: dark = high revenue

Top Categories (Bar Chart):

- Y-axis: `orders_fact[category_en]`
- X-axis: `[Total Revenue]`
- Sort descending
- Top N filter: Top 15 by Revenue

RFM Segment Distribution (Donut/Bar Chart):

- Axis: `rfm_segments[segment]`
- Values: count of `customer_unique_id`
- Use the SEGMENT_COLORS palette from Phase 8

Step 6: Add Slicers

- Date Range slicer: `DateTable[Date]` → Slicer → Between
- Customer State slicer: `orders_fact[customer_state]` → Slicer → Dropdown
- Category slicer: `orders_fact[category_en]` → Slicer → List

Step 7: Format for Professionalism

- Set page background to `#F8F9FA` (light gray — easier on the eyes than pure white)
- Use a consistent 2-color palette: `#2C4A8A` (deep blue) and `#E07B39` (accent orange)
- Remove borders from all cards
- Align all visuals to a grid (View → Gridlines → Snap to Grid)
- Use consistent font: Segoe UI, size 11 for labels, 14 for titles
- Add a title text box: "Olist E-Commerce Analytics Dashboard | 2017-2018"

Tableau Equivalent Notes

If using Tableau Public instead of Power BI:

- Import CSVs via Data Source tab
- Create a Date hierarchy by right-clicking the timestamp field
- KPI cards: use "Show Me" → Text table with a single measure
- Cohort heatmap: Rows = cohort_month, Columns = period, Color = retention_rate (square

marks)

- Use Calculated Fields for all your KPI formulas
- Publish to Tableau Public for a shareable URL to include in your GitHub README

What This Project Demonstrates to Recruiters

At the end of Phases 1-11, your project demonstrates:

Skill	Where It Shows
SQL (joins, CTEs, window functions)	Phase 6 — 20 business queries
ETL pipeline design	Phase 4 — modular Python ETL
Business thinking	Phases 5, 9 — analyst-framed EDA and KPIs
Cohort analysis	Phase 7 — retention heatmap
Customer segmentation	Phase 8 — RFM with segment strategy
Dashboard design	Phase 11 — Power BI with DAX measures
Python/pandas	Throughout
Data modeling	Phase 3 — schema design

The combination of ML/research background + this analytics project signals you can do both technical modeling *and* business-facing analysis — a profile that is genuinely rare among undergraduate candidates and makes you competitive for analyst roles at tech companies, consulting firms, and data-forward businesses.

Built from the Olist Brazilian E-Commerce dataset (Kaggle). 99,441 customers · 9 tables · 18 months of transactions.